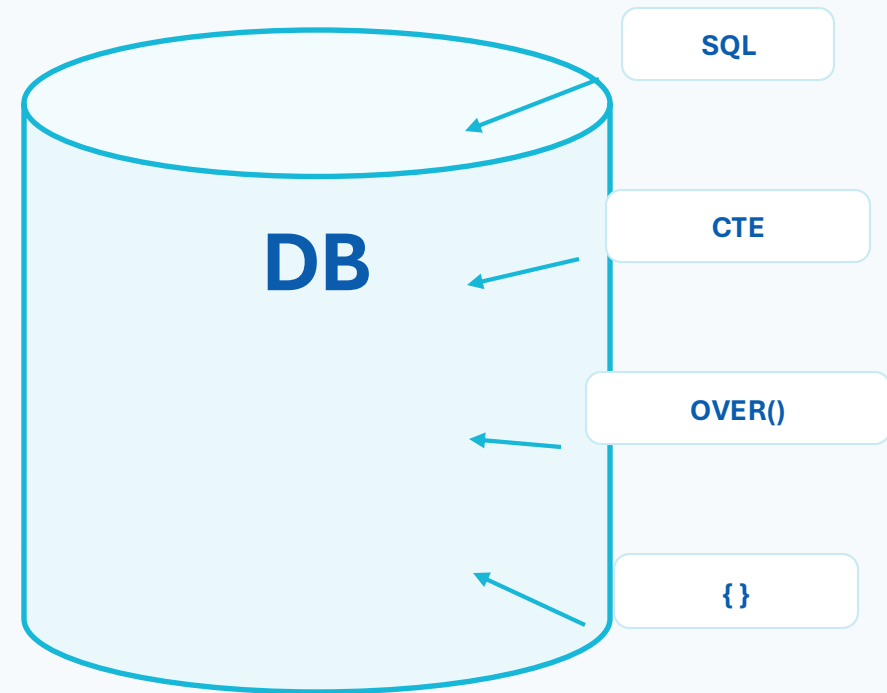


Sistemas de Gestión de Bases de Datos

Unidad 3 + 4 · SQL avanzado y programabilidad

De consultas expresivas a reglas y evolución controlada dentro del motor

MySQL · PostgreSQL · 2026



Objetivo de las unidades

Qué debería poder decidir y justificar al finalizar este bloque

Objetivo de aprendizaje

Resolver consultas complejas, implementar reglas y automatismos dentro del motor y evolucionar una base existente mediante cambios verificables y controlados.



Consultar

Resolver requerimientos complejos sin perder la intención del resultado.



Analizar

Elegir entre alternativas y verificar su comportamiento.



Programar

Implementar reglas y automatismos apropiados.



Evolucionar

Modificar el esquema sin perder control sobre los datos.

No buscamos memorizar sintaxis: buscamos elegir dónde y cómo resolver un problema.

Pregunta disparadora

Un único escenario para recorrer SQL avanzado, programabilidad y migraciones

Tenemos CLIENTE, VENTA, PRODUCTO y PEDIDO. El sistema creció y ahora necesita reportes, reglas de negocio y una nueva versión de la base.

1 SQL

¿Cómo obtengo la última venta de cada cliente?

2 SQL

¿Cómo calculo acumulados sin perder detalle?

3 Diseño

¿Dónde debería controlar una regla de negocio?

4 Estrategía

¿Cómo modifico el esquema sin destruir información?

La misma base nos permite conectar consulta, comportamiento y evolución.

Mapa de contenido

Teoría breve + ejemplos en motor + decisiones justificadas

1 Consultas complejas

Subconsultas,
EXISTS y joins.

2 Ventanas y CTE

Acumulados, ranking
y etapas.

3 Jerarquías

Recursividad y
profundidad variable.

4 Programabilidad

Constraints,
funciones,
procedimientos y
triggers.

5 Evolución

Migraciones,
rollback, pruebas y
CI/CD.

SQL avanzado resuelve información · la programabilidad agrega comportamiento · las migraciones permiten evolucionar la solución.



PARTE 1: SQL AVANZADO

¿Qué significa realmente SQL avanzado?

No es “más comandos”: es expresar mejor un requerimiento complejo

SQL avanzado

A Expresiones y datos

Texto, fechas, NULL y agregación sin atarse innecesariamente a un motor.

B Relaciones y conjuntos

JOIN, subconsultas y EXISTS para conservar o excluir exactamente las filas correctas.

C Análisis y estructura

Ventanas, CTE y recursividad para secuencias, ranking, acumulados y jerarquías.

La pregunta guía ya no es “¿qué sintaxis existe?”, sino “¿qué resultado necesito preservar?”.

Antes de escribir SQL: definir el resultado

Caso guía: “obtener la última compra de cada cliente”

“Última venta” parece claro... hasta que aparecen clientes sin ventas y dos ventas en la misma fecha.

1 Cobertura

¿Incluimos clientes sin compras?

2 Empates

¿Una fila o todas las compras de la fecha máxima?

3 Desempate

¿Qué criterio define una única fila?

4 Salida

¿Qué columnas debe conservar el reporte?

Muchas consultas incorrectas son sintácticamente válidas.

Valores, texto y fechas: expresiones portables

Usar SQL estándar cuando resuelve el problema de forma clara



NULL

La comparación puede producir UNKNOWN. Para probar ausencia se usa IS NULL / IS NOT NULL.



Texto

CONCAT y expresiones de cadena permiten construir etiquetas o rutas.



Fechas

EXTRACT permite recuperar año o mes de manera portable.

```
SELECT
  COALESCE(segmento, 'SIN_SEGMENTO') AS segmento,
  CONCAT(nombre, ' - ', ciudad) AS etiqueta,
  EXTRACT(YEAR FROM fecha) AS anio
FROM cliente;
```

Primero portabilidad y claridad; después, funciones específicas del motor si aportan valor.

JOIN: el resultado depende de qué queremos conservar

No memorizar diagramas: elegir la semántica correcta



INNER JOIN

Conserva clientes que encuentran una fila relacionada en VENTA.



LEFT JOIN

Conserva todos los clientes, incluso si no tienen ventas.



NOT EXISTS

Permite expresar un anti-join: clientes para los que no existe una venta.

Antes de elegir JOIN, decidir qué filas deben sobrevivir al resultado.

Subconsulta, JOIN o EXISTS

Distintas formas pueden ser lógicamente equivalentes sin ser idénticas

Cientes con al menos una venta mayor a 100.000

1

Subconsulta

Útil cuando la consulta interna produce un conjunto o un valor que alimenta la consulta externa.

2

JOIN

Natural cuando necesitamos combinar columnas de ambas relaciones.

3

EXISTS

Natural cuando sólo importa saber si existe al menos una fila relacionada.

El optimizador decide el plan; nosotros comparamos intención, legibilidad y evidencia.

EXISTS: preguntar por existencia, no por cantidad

Cuando el resultado sólo necesita saber si hay una fila relacionada

```
SELECT c.cliente_id, c.nombre
FROM cliente c
WHERE EXISTS (
    SELECT 1
    FROM venta v
    WHERE v.cliente_id = c.cliente_id
        AND v.total > 100000
);
```



Lectura

“Devuelva el cliente si existe al menos una venta que cumpla la condición”.



Variación

NOT EXISTS expresa el caso contrario: clientes que nunca tuvieron una venta.

EXISTS comunica intención y deja al optimizador decidir cómo verificarla.

GROUP BY cambia la granularidad

Una agregación clásica reduce filas: eso a veces es exactamente lo que no queremos

venta	cliente	total
V1	A	100
V2	A	80
V3	B	50
V4	B	70

4 filas



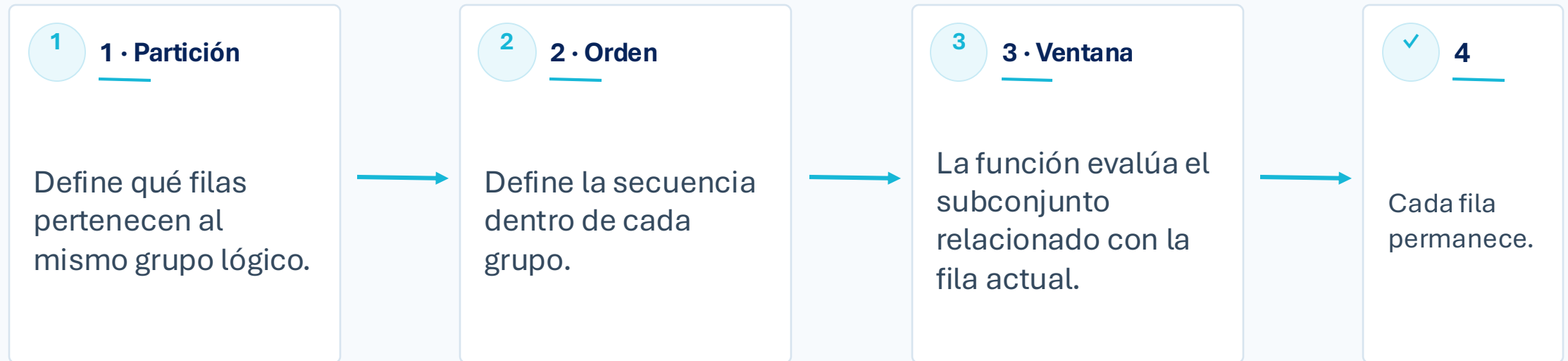
cliente	SUM(total)
A	180
B	120

2 filas

¿Cómo calculamos un total o acumulado y, al mismo tiempo, conservamos cada venta?

Funciones de ventana

Calcular sobre filas relacionadas sin perder granularidad



GROUP BY resume; OVER() agrega contexto analítico a cada fila.

Anatomía de OVER()

Tres decisiones separadas: grupo, orden y marco

cliente_id	venta_id	total	Total_Ventas <i>(Resultado del query)</i>
1	101	\$50	\$50 <i>(Primera venta)</i>
1	102	\$30	\$80 <i>(\$50 + \$30)</i>
1	103	\$20	\$100 <i>(\$80 + \$20)</i>

El frame es especialmente importante en acumulados y promedios móviles.

Ejemplo: acumulado mensual conservando detalle

La fila sigue representando una venta; el acumulado se agrega como una columna analítica

```
SUM(v.total) OVER (  
  PARTITION BY v.cliente_id,  
               EXTRACT(YEAR FROM v.fecha),  
               EXTRACT(MONTH FROM v.fecha)  
  ORDER BY v.fecha, v.venta_id  
  ROWS BETWEEN UNBOUNDED PRECEDING  
             AND CURRENT ROW  
) AS acumulado_mes
```



Cliente 2 · septiembre

120.000 → 205.000 → 300.000



Detalle preservado

Cada venta conserva fecha, total, canal y demás columnas.

ROWS + un segundo criterio de orden evita ambigüedad cuando hay empates de fecha.

Ranking: elegir qué significa un empate

ROW_NUMBER, RANK y DENSE_RANK no expresan la misma decisión

cliente	total	ROW_NUMBER	RANK	DENSE_RANK
A	300	1	1	1
B	200	2	2	2
C	200	3	2	2
D	100	4	4	3

1 ROW_NUMBER

Siempre asigna un número diferente.

2 RANK

Comparte posición y deja huecos.

3 DENSE_RANK

Comparte posición sin dejar huecos.

La función correcta depende de la semántica del ranking, no de una preferencia de sintaxis.

Caso típico: la última fila de cada grupo

Ventana + filtro para elegir una fila completa, no sólo la fecha máxima

```
WITH ventas_ordenadas AS (  
    SELECT v.*,  
           ROW_NUMBER() OVER (  
               PARTITION BY cliente_id  
               ORDER BY fecha DESC, venta_id  
           ) AS rn  
    FROM venta v  
)  
SELECT *  
FROM ventas_ordenadas  
WHERE rn = 1;
```

1

1 · Particionar

Cada cliente genera su propio orden.

2

2 · Desempatar

fecha DESC + venta_id DESC define una fila.

3

3 · Filtrar

rn = 1 conserva la fila elegida.

CTE: poner nombre a una etapa de la consulta

Una consulta compleja se vuelve más legible cuando el razonamiento queda explícito



Una Common Table Expression o CTE dura lo que dura la sentencia: organiza el SQL, no crea una tabla persistente.

Ejemplo: totalizar primero, rankear después

Dos niveles lógicos se expresan mejor como dos etapas

```
WITH totales AS (  
    SELECT c.cliente_id, c.segmento,  
           COALESCE(SUM(v.total),0) AS total_vendido  
    FROM cliente c  
    LEFT JOIN venta v ON v.cliente_id = c.cliente_id  
    GROUP BY c.cliente_id, c.segmento  
)  
SELECT cliente_id, segmento, total_vendido,  
       DENSE_RANK() OVER (  
           PARTITION BY segmento  
           ORDER BY total_vendido DESC  
       ) AS posicion  
FROM totales;
```

1

Etapa 1

Calcular total por cliente.

2

Etapa 2

Rankear dentro de cada segmento.

CTE vs. subconsulta derivada

Elegir la forma que haga visible la estructura del problema

```
SELECT ...  
FROM (  
    SELECT ...  
    FROM ...  
) etapa;
```

Subconsulta derivada

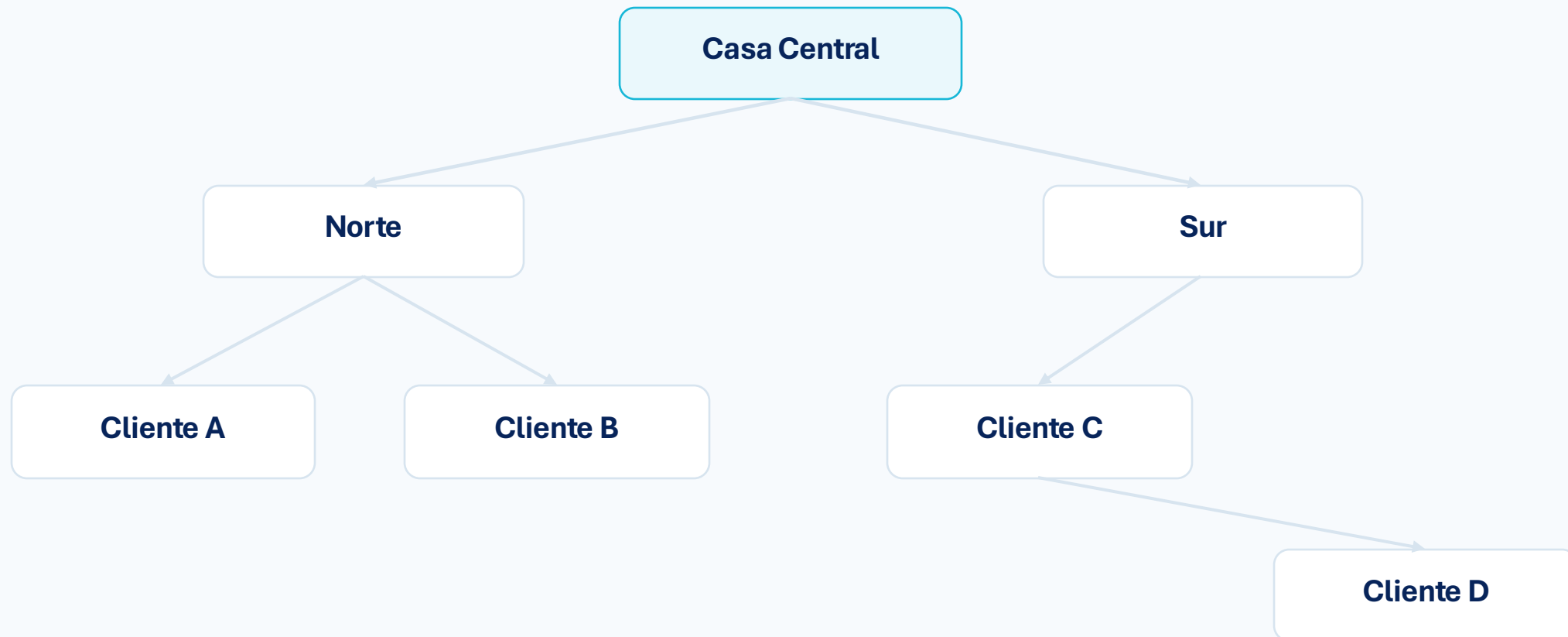
```
WITH etapa AS (  
    SELECT ...  
    FROM ...  
)  
SELECT ...  
FROM etapa;
```

CTE con nombre

No asumir que una CTE “siempre es más rápida”: revisar el plan cuando el rendimiento importe.

Problema: jerarquías de profundidad desconocida

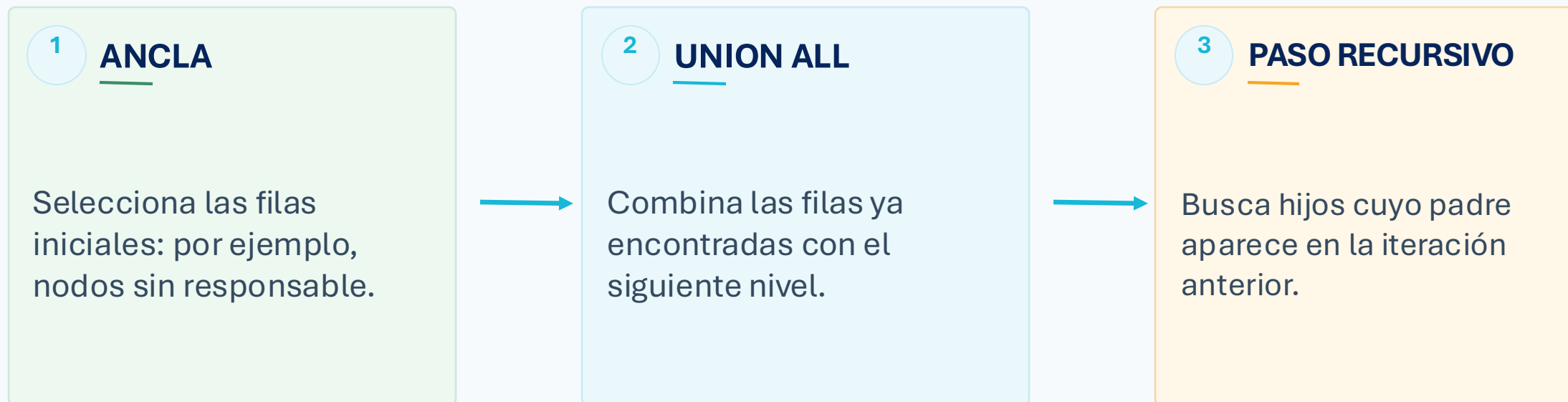
Un número fijo de self-joins deja de servir cuando el árbol puede crecer



¿Cuántos JOIN necesitamos si no sabemos la profundidad?

Anatomía de una CTE recursiva

Ancla + paso recursivo + condición de terminación implícita



La consulta termina cuando una iteración ya no produce nuevas filas.

Ejemplo jerárquico: mismo patrón, pequeñas diferencias

La lógica es común; la construcción de la ruta cambia ligeramente entre motores

```
WITH RECURSIVE jerarquia AS (  
    SELECT cliente_id, nombre, responsable_id,  
           0 AS nivel, <ruta_inicial> AS ruta  
    FROM cliente  
    WHERE responsable_id IS NULL  
  
    UNION ALL  
  
    SELECT c.cliente_id, c.nombre, c.responsable_id,  
           j.nivel + 1, <ruta_siguiente>  
    FROM cliente c  
    JOIN jerarquia j  
        ON c.responsable_id = j.cliente_id  
)  
SELECT * FROM jerarquia;
```



PostgreSQL

Ruta inicial: nombre::text

Ruta siguiente: j.ruta || '>' ||
c.nombre



MySQL

Conviene CAST(nombre AS
CHAR(...)) y luego CONCAT(j.ruta, '>
' , c.nombre).

¿Cuándo usar recursividad?

No toda autorreferencia necesita una CTE recursiva



Sí tiene sentido

- Organigramas y jerarquías
- Categorías/subcategorías
- BOM / desglose de materiales
- Dependencias y árboles de permisos
- Profundidad variable o desconocida



No necesariamente

- Profundidad conocida y muy pequeña
- Datos que conviene precalcular
- Tabla de cierre o path como decisión deliberada
- Recorrido que pertenece mejor a otra capa

La recursividad es una herramienta para expresar estructura variable, no un fin en sí misma.

Carga masiva: una fila a la vez vs. operación por lote

Cuando el volumen crece, el costo por interacción importa

1× 20.000 INSERT individuales

Aplicación → motor

Aplicación → motor

Aplicación → motor

...

20.000 interacciones y más overhead por fila.



N× Operación bulk

Archivo / lote



comando de carga masiva



tabla

Menos overhead por fila.

La herramienta cambia por motor; el principio es minimizar el costo repetido por fila.

Carga masiva en los dos motores

La sintaxis cambia; el objetivo operativo es el mismo

MySQL

```
LOAD DATA LOCAL INFILE 'ventas.csv'  
INTO TABLE venta  
FIELDS TERMINATED BY ','  
IGNORE 1 LINES;
```

PostgreSQL

```
COPY venta  
FROM '/ruta/ventas.csv'  
WITH (FORMAT csv, HEADER true);
```

Antes de cargar: definir formato, tipos, manejo de errores, transacción y validación posterior.

Resumen · Parte 1: SQL avanzado

Qué decisiones y herramientas deberían quedar incorporadas

1 Semántica del resultado

Definir cobertura, empates, desempate y columnas antes de escribir SQL.

La consulta puede ser válida y, aun así, responder otra pregunta.

Primero: ¿qué filas deben sobrevivir?

2 Relaciones y existencia

JOIN para combinar datos.

LEFT JOIN para preservar filas sin relación.

EXISTS / NOT EXISTS cuando importa la existencia.

Elegir por intención, no por costumbre.

3 Ventanas

OVER() permite agregar contexto analítico sin perder detalle.

PARTITION BY, ORDER BY y frame.

Acumulados, ranking y última fila por grupo.

GROUP BY resume · OVER() conserva filas.

4 CTE y recursividad

CTE para expresar consultas por etapas.

CTE recursiva para jerarquías de profundidad variable.

No asumir mejoras de rendimiento sin medir.

Hacer visible la estructura del problema.

5 Carga masiva

Carga por lote para reducir overhead por fila.

MySQL: LOAD DATA
PostgreSQL: COPY

Mismo principio, distinta sintaxis.

Idea integradora: SQL avanzado no es “más sintaxis”; es elegir una construcción que preserve correctamente la semántica del resultado.



PARTE 2: PROGRAMABILIDAD

¿Dónde debería vivir la lógica de negocio?

Misma regla, distintas ubicaciones posibles. ¿Qué priorizamos: flexibilidad, reutilización o integridad?

Aplicación

Conviene cuando...

- Experiencia de usuario
- Reglas específicas del canal

Pregunta

¿Cómo evitamos duplicarla en otros canales?

Capa media / servicios

Conviene cuando...

- Regla compartida por varios canales
- Orquestación de procesos

Pregunta

¿Qué ocurre si alguien accede por fuera del servicio?

Base de datos

Conviene cuando...

- Integridad que debe cumplirse siempre
- Regla cercana al dato

Pregunta

¿Cuánto acoplamiento al motor aceptamos?

DEBATE: “Una transferencia no puede superar el límite diario o el saldo no puede ser negativo”.
¿Dónde pondrían esa regla y por qué?

Mapa de mecanismos

Elegir primero el mecanismo más simple que garantice correctamente la regla



Necesidad	Mecanismo candidato
Validar una fila	CHECK / constraint
Calcular y reutilizar	Función
Ejecutar un proceso explícito	Procedimiento
Reaccionar automáticamente	Trigger
Cambiar la estructura	Migración de esquema

Poder hacerlo con un trigger no significa que debemos hacerlo con un trigger.

Integridad declarativa antes que código

Una regla simple sobre una fila suele expresarse mejor con CHECK

```
ALTER TABLE cuenta  
ADD CONSTRAINT  
chk_cuenta_saldo  
CHECK (saldo >= 0);
```



Válido

saldo = 1000
✓ acepta



Límite

saldo = 0
✓ acepta



Inválido

saldo = -1
✗ rechaza

Si la regla puede declararse directamente, evitamos código procedimental innecesario.

Función vs. procedimiento

¿Necesitamos un valor reutilizable o ejecutar un proceso explícito?



FUNCIÓN

- Devuelve un resultado
- Puede reutilizarse como expresión
- Útil para cálculos y reglas de clasificación
- Conviene mantenerla pequeña y predecible



PROCEDIMIENTO

- Se invoca explícitamente
- Coordina varios pasos
- Puede recibir parámetros
- Puede modificar datos y encapsular una operación

La diferencia útil para decidir es semántica: valor vs. proceso.

Ejemplo de Función: clasificar un cliente por total anual

Primero definir la regla; después traducirla al lenguaje del motor

BRONCE

< 2M

PLATA

2M – 50M

ORO

> 50M

```
clasificar_cliente(cliente_id, anio) → categoría
```

Los umbrales son parte del requerimiento; la función encapsula una decisión reusable.

Programación dentro del motor

Conceptos comunes aunque la sintaxis cambie entre MySQL y PostgreSQL

1 Parámetros

Entradas y salidas

2 Variables

Estado local

3 IF / CASE

Decisión

4 SQL

Lectura y escritura

5 Errores

Detectar y responder

6 Transacción

Coherencia del proceso

Primero comprender la lógica; después implementar la sintaxis de cada motor.

Stored procedure · mismo problema, dos motores

Caso: encapsular una operación explícita que cambia el estado de un pedido

MySQL

```
DELIMITER //  
CREATE PROCEDURE cambiar_estado_pedido(  
    IN p_pedido_id INT,  
    IN p_estado VARCHAR(12)  
)  
BEGIN  
    UPDATE pedido  
    SET estado = p_estado  
    WHERE pedido_id = p_pedido_id;  
END//  
DELIMITER ;  
  
CALL cambiar_estado_pedido(10, 'pagado');
```

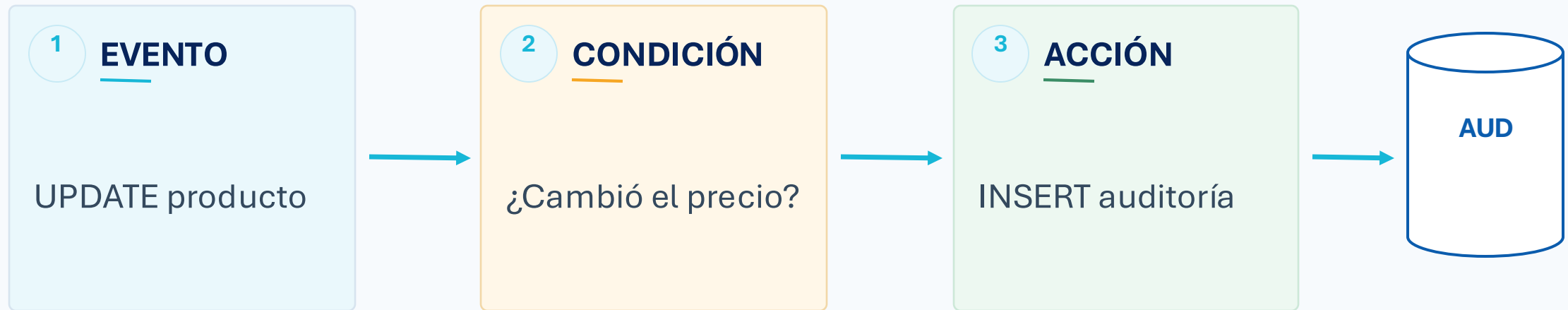
PostgreSQL

```
CREATE PROCEDURE cambiar_estado_pedido(  
    p_pedido_id INTEGER,  
    p_estado VARCHAR(12)  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    UPDATE pedido  
    SET estado = p_estado  
    WHERE pedido_id = p_pedido_id;  
END;  
$$;  
  
CALL cambiar_estado_pedido(10, 'pagado');
```

Qué observar: el objetivo es el mismo; cambia la sintaxis. El procedimiento se invoca explícitamente con CALL y concentra una operación reutilizable.

Trigger: reacción automática

Evento → condición → acción dentro de la misma operación



Un trigger no se llama: el motor lo ejecuta cuando ocurre el evento configurado.

BEFORE / AFTER y OLD / NEW

Los valores disponibles dependen del evento que dispara el trigger

Evento	OLD	NEW
INSERT	—	✓
UPDATE	✓	✓
DELETE	✓	—

B BEFORE

Útil para validar o transformar antes de persistir.

A AFTER

Útil para registrar o reaccionar una vez aplicada la modificación.

En auditoría de UPDATE, OLD y NEW permiten comparar el valor anterior con el nuevo.

Trigger · auditoría automática de cambios

Caso: registrar precio anterior y nuevo cuando cambia el precio de un producto

MySQL · lógica directa en el trigger

```
DELIMITER //  
CREATE TRIGGER trg_precio_aud  
AFTER UPDATE ON producto  
FOR EACH ROW  
BEGIN  
    IF OLD.precio <> NEW.precio THEN  
        INSERT INTO auditoria_precio  
            (producto_id, precio_anterior, precio_nuevo)  
        VALUES  
            (OLD.producto_id, OLD.precio, NEW.precio);  
    END IF;  
END//  
DELIMITER ;
```

PostgreSQL · función + trigger

```
CREATE FUNCTION fn_precio_aud()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF OLD.precio IS DISTINCT FROM NEW.precio THEN  
        INSERT INTO auditoria_precio  
            (producto_id, precio_anterior, precio_nuevo)  
        VALUES  
            (OLD.producto_id, OLD.precio, NEW.precio);  
    END IF;  
    RETURN NEW;  
END; $$ LANGUAGE plpgsql;  
  
CREATE TRIGGER trg_precio_aud  
AFTER UPDATE ON producto  
FOR EACH ROW EXECUTE FUNCTION fn_precio_aud();
```

Diferencia clave: en MySQL la lógica se escribe dentro del trigger; en PostgreSQL el trigger invoca una función. En ambos casos se ejecuta automáticamente dentro de la transacción que causó el UPDATE.

Constraint , función , procedimiento o trigger

Elegir por responsabilidad, no por “potencia” del mecanismo

A

Caso A · saldo ≥ 0

Regla simple, local a la fila y declarativa.

Preferencia: CHECK

La base rechaza cualquier operación que viole la condición.

B

Caso B · auditar precio

Necesita reaccionar ante un UPDATE y guardar valores anterior/nuevo.

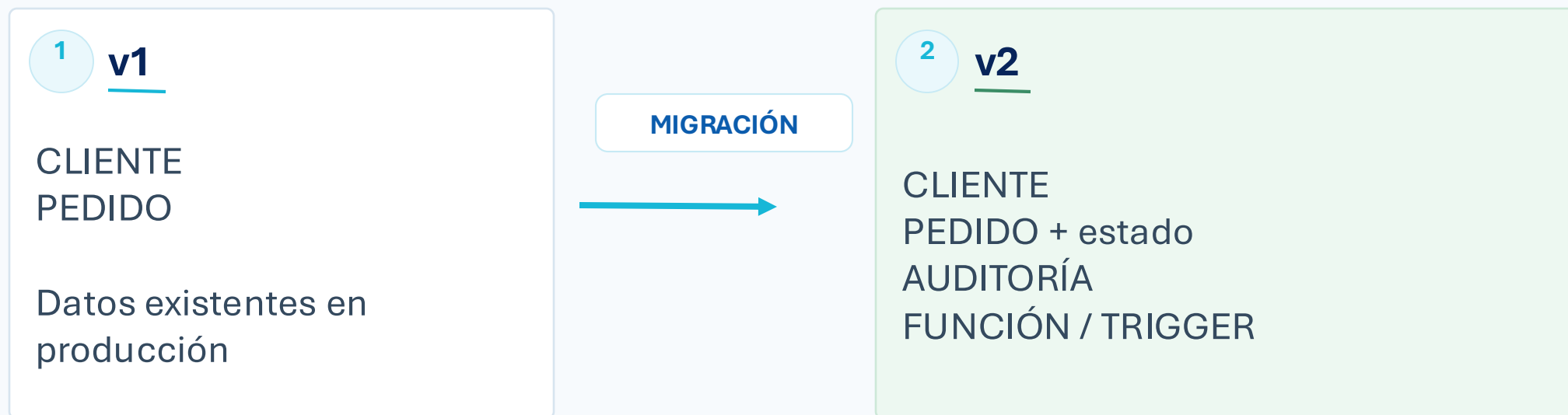
Preferencia: TRIGGER

La acción ocurre automáticamente.

La regla más simple debe implementarse con el mecanismo más simple que la garantice.

Una base de datos también tiene versiones

El esquema y sus objetos evolucionan junto con la aplicación



Modificar el esquema no debería implicar recrear la base ni perder datos.

Migrar no es recrear

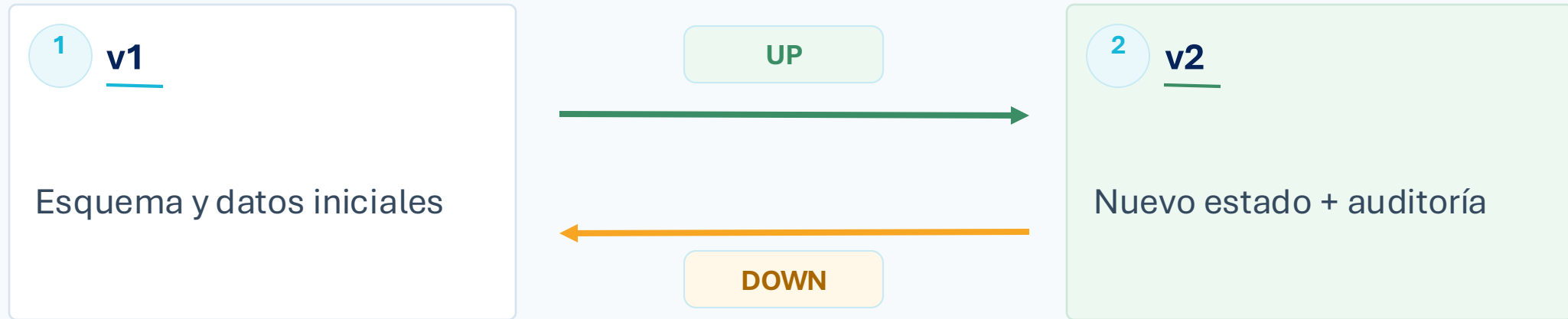
Secuencia segura para introducir una nueva columna obligatoria



Expandir → completar históricos → validar → endurecer → habilitar lógica.

Forward y rollback

Revertir estructura no siempre recupera la información generada en la versión nueva



Advertencia: DROP COLUMN estado puede destruir información creada en v2. Un rollback seguro puede requerir copia, backup o migración inversa.

Scripts de migración versionados

Una release de base de datos debe ser ordenada, trazable y repetible

```
V001__modelo_inicial.sql  
V002__estado_pedido.sql  
V003__auditoria_estado.sql  
V004__funcion_total_cliente.sql
```

1

Orden

La versión indica qué cambio debe aplicarse antes.

2

Trazabilidad

Cada script explica qué cambió y cuándo.

3

Repetibilidad

El despliegue debe ser reproducible en distintos ambientes.

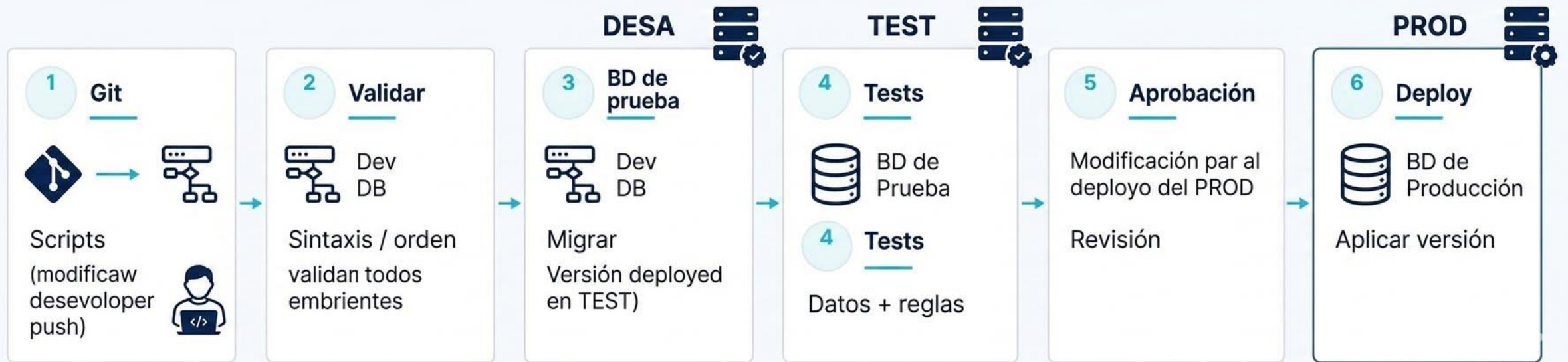
Versionar la base es tratar su evolución como parte del software.

De migración a CI/CD

Automatizar controles antes de que el cambio llegue a producción

Flujo de CI/CD para Base de Datos

Visualizando el progreso de la base de datos a través de DESA, TEST y PROD



CI/CD no elimina el riesgo: hace repetibles las validaciones y el despliegue.

¿Dónde resolver cada problema?

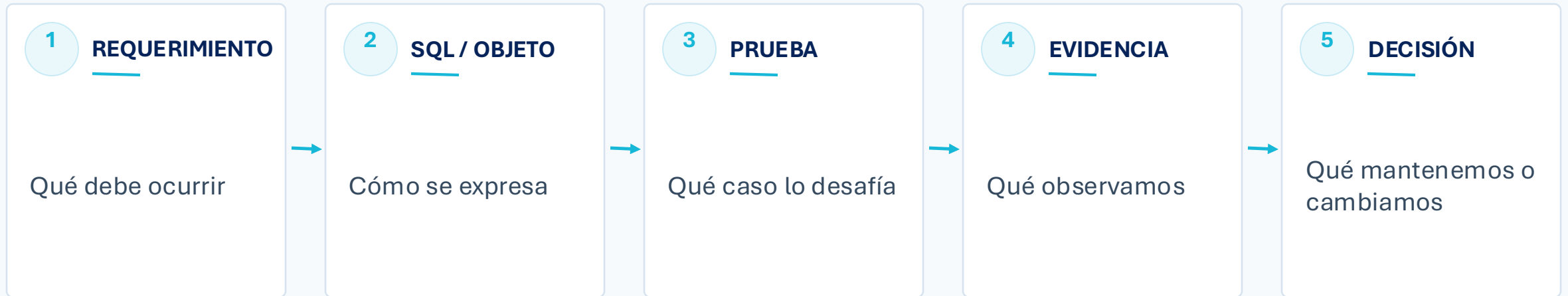
Una matriz de decisión para integrar SQL avanzado, programabilidad y migraciones

Problema	Herramienta probable
Relacionar y filtrar	JOIN / EXISTS
Mantener detalle + agregado	Función de ventana
Dividir una consulta en etapas	CTE
Recorrer una jerarquía	CTE recursiva
Regla simple sobre una fila	Constraint
Cálculo reusable	Función
Proceso explícito	Procedimiento
Reacción automática	Trigger
Evolucionar estructura	Migración

La herramienta es consecuencia del problema que queremos expresar y garantizar.

El criterio profesional

Una solución de base de datos debe ser correcta, verificable y mantenible



Que funcione es necesario. Poder explicar por qué funciona y cómo evolucionará es lo que convierte la solución en ingeniería.

Resumen · Parte 2: Programabilidad y evolución

Cómo decidir dónde poner la lógica y cómo hacer evolucionar la base

1

Ubicar la lógica

Aplicación: experiencia y reglas de canal.

Capa media: reglas compartidas y orquestación.

BD: integridad que debe cumplirse siempre.

Debate: flexibilidad vs. garantía.

2

Elegir mecanismo

CHECK para regla declarativa.

Función para valor reusable.

Procedimiento para proceso explícito.

Trigger para reacción automática.

Usar el mecanismo más simple suficiente.

3

Programar en el motor

Parámetros, variables, IF/CASE, SQL, errores y transacción.

La sintaxis cambia entre motores; la responsabilidad es la misma.

Comprender la lógica antes del detalle sintáctico.

4

Triggers

Evento → condición → acción.

BEFORE / AFTER.

OLD / NEW para comparar valores.

MySQL: lógica directa.
PostgreSQL: función + trigger.

Automático ≠ mejor por defecto.

5

Evolución

Versionar scripts.

Migrar v1 → v2 sin recrear.

Validar continuidad.

Rollback razonado.

CI/CD para repetir controles.
La BD también es software.

Criterio profesional: requisito → SQL/objeto → prueba → evidencia → decisión. Que funcione es necesario; poder justificarlo y evolucionarlo es ingeniería.



PARTE 3: ACTIVIDADES TIPO DESAFIO

Previa antes de presentar las actividades

Tres preguntas que deberían quedar respondidas

DESAFIO 1



¿Cómo consulto?

Elegir una construcción SQL que preserve correctamente la semántica del resultado.

DESAFIO 2



¿Dónde implemento la regla?

Elegir el mecanismo de base de datos más simple que garantice la responsabilidad.

DESAFIO 3



¿Cómo evoluciono?

Versionar, migrar, validar y poder razonar el rollback.

SQL avanzado + programabilidad + migraciones = una base de datos correcta, verificable y mantenible.

Actividad 1 · El reporte que nadie puede resolver

SQL avanzado · resolver, comparar y defender

ACTIVIDAD 1

SQL Desafío

Resolver dentro del SGBD un reporte analítico sobre clientes, ventas y jerarquías utilizando recursos SQL avanzados.

1 Qué debe aparecer

- Última venta
- Acumulado mensual
- Ranking
- Jerarquía

2 Qué se defiende

- Resultado
- Decisión SQL
- Casos borde
- Evidencia

Producto: script + resultados + explicación técnica.

Actividad 2 · La regla que no puede romperse

Programabilidad · elegir el mecanismo correcto y probarlo

ACTIVIDAD 2

Decisión central

Para cada regla, justificar si corresponde un constraint, una función/procedimiento o un trigger.

Casos

- Saldo no negativo
- Auditoría de precio
- Clasificación anual
- Regla adicional

Evidencia

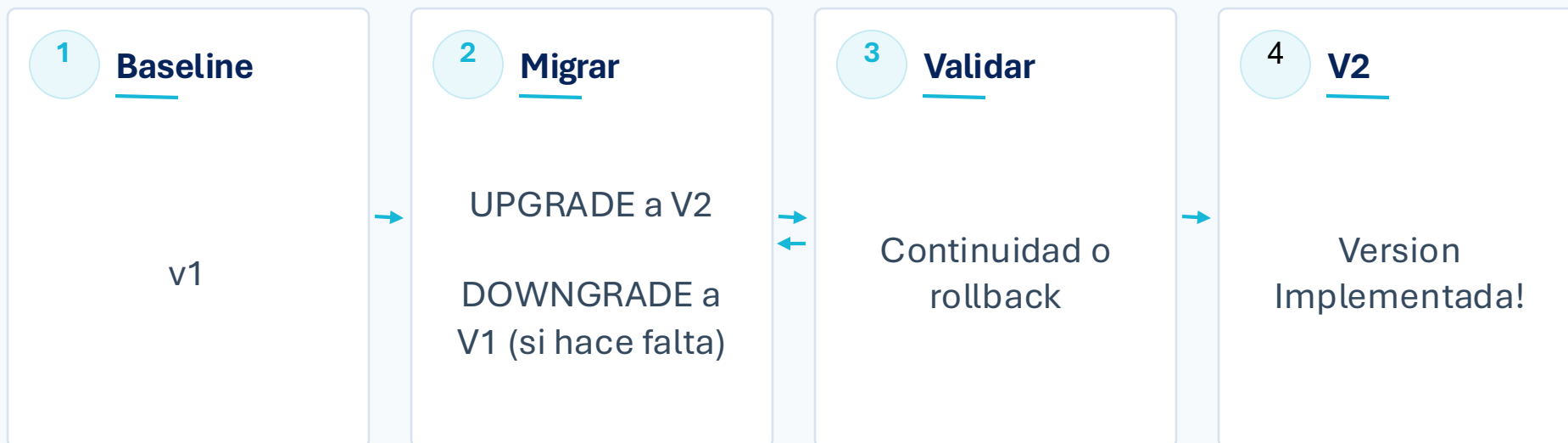
- Caso válido
- Caso límite
- Caso inválido
- Defensa de la elección

Producto: objetos ejecutables + pruebas + defensa.

Actividad 3 · Release de base de datos v2.0

Integración U3 + U4 · migrar sin perder continuidad

ACTIVIDAD 3



Producto: scripts versionados + evidencia de continuidad + rollback razonado.

Una release de BD no termina cuando ejecuta el script: termina cuando podemos demostrar continuidad, funcionamiento y reversibilidad.